Prvi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 12.11.2024.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2024_Test1_GrupaA_IT1g2023**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Merge Sort*

Napisati metodu ***merge*** - ***Merge Sort*** algoritma za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomArrayGenerator*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***MergeSort*** napisati implementaciju za metodu ***private static void merge***(long[] arr, long[] tmpArray, int lowerIndex, int middleIndex, int upperIndex)
- Obezbediti kod u **Main** metodi za testiranje algoritma.

Zadatak 2 – *Spajanje dve jednostruko spregnute liste, rastući poredak*

Spojiti dve prethodno sortirane, jednostruko spregnute liste u jednu, tako da elementi rezultujuće liste budu sortirani u rastućem poretku. Algoritam je neophodno napisati sa linearnom vremenskom kompleksnošću O(n).

Dodatni zahtevi za rešavanje zadatka:

- Algoritam pisati u okviru klase ***LinkedList.***
- Pretpostavimo da su početne liste već sortirane.
- Zahtevan potpis metode: ***public static LinkedList MergingTwoLinkedLists (LinkedList list1, LinkedList list2)***

Primer:

- Input: 1 ↔ 2 ↔ 3 ↔ 4 ; 1 ↔ 2 ↔ 3 ↔ 4
- Output: 1 ↔ 1 ↔ 2 ↔ 2 ↔ 3 ↔ 3 ↔ 4 ↔ 4
- Input: 2 ↔ 5 ↔ 6 ↔ 7 ; 1 ↔ 4 ↔ 8 ↔ 9
- Output: 1 ↔ 2 ↔ 4 ↔ 5 ↔ 6 ↔ 7 ↔ 8 ↔ 9

Zadatak 3 – *Suma brojeva reda jednaka k*

Napišite algoritam koji pronalazi sve uzastopne podnizove elemenata u redu čija je suma jednaka zadanom broju k. Metoda treba da ispiše svaki podniz koji zadovoljava ovaj uslov.

Dodatni zahtevi za rešavanje zadatka:

- Zadatak rešavati pomoću strukture reda (klasa ***Queue***) koja je data u okviru šablona zadatka.
- Zahtevan potpis metode: ***public void FindSubsequencesWithSum(int targetSum)***

Primer1:

- Ulaz za red: 1,4,1,1,5 ; k = 6
- Ispis: Podniz sume 6: 1 4 1 ;
  Podniz sume 6: 4 1 1;
  Podniz sume 6: 1 5

Primer2:

- Ulaz za red: 2, 5, 1, 7, 2; k=8
- Ispis: Podniz sume 8: 2, 5, 1
  Podniz sume 8: 1, 7